

Monitors

Higher-level synchronization with automatic mutual exclusion

Definition

A **monitor** is a high-level synchronization construct that groups procedures, variables, and data structures into one protected module. Only one process or thread can be active inside a monitor at a time.

1. Why Monitors Were Introduced

Main Idea

Semaphores and mutexes are powerful, but they are easy to use incorrectly. A small mistake in the order of operations can cause deadlock or race conditions.

Semaphore Ordering Problem

In the producer-consumer semaphore solution, the producer should do `down(&empty)` before `down(&mutex)`. If the order is reversed and the buffer is full, the producer can block while holding `mutex`.

Deadlock

If the producer blocks while holding `mutex`, the consumer cannot enter the buffer to remove an item. The consumer also blocks, and both processes wait forever. This is called **deadlock**.

Key Motivation

Monitors reduce programmer error by moving mutual exclusion into the language or runtime system instead of forcing programmers to manually place every lock and unlock correctly.

2. What a Monitor Contains

Monitor Structure

A monitor is like a special module or package containing:

- private variables,
- private data structures,
- procedures that operate on those variables and data structures,
- condition variables for blocking and signaling.

Processes may call monitor procedures, but they cannot directly access the monitor's internal data from outside the monitor.

```

monitor example
  integer i;
  condition c;

  procedure producer();
    ...
  end;

  procedure consumer();
    ...
  end;
end monitor;

```

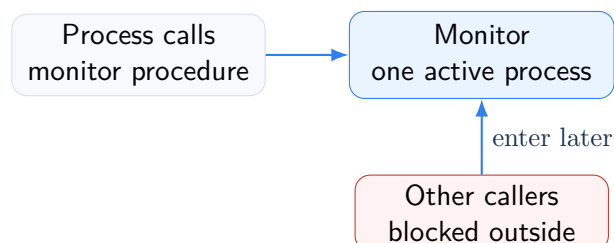
3. Automatic Mutual Exclusion

Monitor Entry Rule

Only one process can be active inside a monitor at any instant.

When a process calls a monitor procedure, the generated code checks whether another process is already active inside the monitor:

- if the monitor is free, the process enters;
- if another process is active, the caller is suspended until the monitor becomes free.



Compiler or Runtime Responsibility

The compiler or runtime arranges the mutual exclusion, often using a mutex or binary semaphore internally. The programmer writes monitor procedures and relies on the monitor rule.

4. Condition Variables in Monitors

Condition Variable

A **condition variable** lets a process inside a monitor block until some condition becomes true.

Mutual exclusion alone is not enough. In producer-consumer, the producer must wait if the buffer is full, and the consumer must wait if the buffer is empty.

Operation	Meaning
<code>wait(condition)</code>	Block the caller and release the monitor so another process can enter.
<code>signal(condition)</code>	Wake one process waiting on that condition, if any exists.

Why Wait Releases the Monitor

If a process waited while still occupying the monitor, no other process could enter to change the condition. Releasing the monitor during `wait` is essential.

5. What Happens After Signal?

The Issue

If one process signals a condition variable and wakes another process, the monitor must still obey the rule that only one process is active inside it at a time.

Proposal	Rule After <code>signal</code>
Hoare	The newly awakened process runs immediately; the signaler is suspended.
Brinch Hansen	A process may signal only as the final action before leaving the monitor.
Common practical variant	The signaler continues, and the awakened process runs after the signaler exits the monitor.

This Note's Simplified Rule

The Brinch Hansen rule is conceptually simple: signal at the end of the monitor procedure, then leave.

6. Condition Variables Have No Memory

Lost Signal

Condition variables are not counters. If `signal` is executed when no process is waiting, the signal is lost.

Difference from Semaphores

Semaphores remember events by incrementing their value. Condition variables do not remember signals. The program must track state with ordinary variables such as `count`.

Rule of Use

A process should test the shared state before waiting. The condition variable only provides the blocking mechanism; the real condition is stored in monitor variables.

7. Producer-Consumer with Monitors

Goal

Use a monitor to protect the buffer automatically and use condition variables to block when the buffer is full or empty.

```
monitor ProducerConsumer
  condition full, empty;
  integer count;

  procedure insert(item: integer);
  begin
    if count = N then wait(full);
    insert_item(item);
    count := count + 1;
    if count = 1 then signal(empty);
  end;

  function remove: integer;
  begin
    if count = 0 then wait(empty);
    remove := remove_item();
    count := count - 1;
    if count = N - 1 then signal(full);
  end;

  count := 0;
end monitor;
```

Condition	Meaning
full	Producer waits here when the buffer has no empty slots.
empty	Consumer waits here when the buffer has no items.
count	Tracks how many items are currently in the buffer.

8. Why Monitors Avoid the Sleep-Wakeup Race

Crucial Difference

Ordinary **sleep** and **wakeup** had a race because one process could try to sleep while another tried to wake it. Inside a monitor, that interleaving cannot happen.

Monitor Protection

If the producer is inside the monitor and discovers that the buffer is full, the consumer cannot enter the monitor until the producer has completed the **wait** operation and is marked as blocked.

Result

The dangerous gap between checking the condition and going to sleep is removed by the monitor's automatic mutual exclusion.

9. Java-Style Monitors

Java Support

Java does not provide classical Hoare-style monitors exactly, but every object can act like a monitor when methods are declared **synchronized**.

Synchronized Methods

If one thread is executing a **synchronized** method of an object, no other thread may execute another **synchronized** method of that same object at the same time.

Java uses `wait()`, `notify()`, and `notifyAll()` for condition-style waiting and signaling inside synchronized code.

10. Java Producer-Consumer Sketch

```
class ProducerConsumerMonitor {
    private final int[] buffer;
    private int count = 0;
    private int lo = 0;
    private int hi = 0;

    ProducerConsumerMonitor(int size) {
        buffer = new int[size];
    }

    public synchronized void insert(int value)
        throws InterruptedException {
        while (count == buffer.length)
            wait();

        buffer[hi] = value;
        hi = (hi + 1) % buffer.length;
        count++;
        notifyAll();
    }

    public synchronized int remove()
        throws InterruptedException {
        while (count == 0)
            wait();

        int value = buffer[lo];
        lo = (lo + 1) % buffer.length;
        count--;
        notifyAll();
        return value;
    }
}
```

Why while Is Used

The waiting condition is checked in a **while** loop because a thread may wake up even though the condition it needs is still false. Rechecking protects correctness.

11. How the Java Monitor Works

Variable / Method	Purpose
buffer	Stores produced items.
count	Counts how many items are currently in the buffer.
lo	Index of the next item to remove.
hi	Index of the next slot to insert into.
insert	Synchronized producer-side monitor procedure.
remove	Synchronized consumer-side monitor procedure.

Safety

When the producer is inside **insert**, the consumer cannot be inside **remove** for the same monitor object. This makes buffer updates safe.

12. Monitors vs. Semaphores

Feature	Semaphores	Monitors
Level	Low-level synchronization primitive.	Higher-level language or runtime construct.
Mutual exclusion	Programmer must use down and up correctly.	Automatic on monitor procedure entry.
Common errors	Wrong order can cause deadlock. Missing operation can cause races.	Fewer locking errors, but condition logic can still be wrong.
Language support	Can be added through system calls or libraries.	Requires language or runtime support.

13. Limitations of Monitors

Language Dependence

Monitors are a programming-language concept. Languages such as C do not have classical monitors built in, so the compiler cannot automatically enforce monitor rules.

Shared-Memory Assumption

Monitors and semaphores assume that processes or threads can share memory. They are not enough for distributed systems where each machine has its own private memory.

Distributed Systems Need More

When computers communicate over a network rather than through shared memory, synchronization primitives alone do not exchange information. Message-passing mechanisms are needed.

14. Quick Review

Key Points

- Monitors were introduced to reduce errors caused by manual semaphore use.
- A monitor groups shared data and procedures into one protected module.
- Only one process can be active inside a monitor at a time.
- Monitor mutual exclusion is handled by the compiler or runtime.
- Condition variables let processes block inside monitors.
- `wait` blocks the caller and releases the monitor.
- `signal` wakes one process waiting on a condition variable.
- Condition variables have no memory, unlike semaphores.
- Java's `synchronized` methods provide monitor-like behavior.
- Monitors require language/runtime support and are mainly for shared-memory systems.